# CHRONIC KIDNEY DISEASE PREDICTION USING MACHINE LEARNING ON AWS CLOUD

**CLOUD COMPUTING**


Submitted by

**AADITYA PARTHA SARATHY – 230701001**

**AKSHAY KUMAR S – 230701021**

**ASHISH P SHAJI – 230701041**


*in partial fulfilment for the award of the degree of*

**BACHELOR OF ENGINEERING**

in

**COMPUTER SCIENCE AND ENGINEERING**


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**RAJALAKSHMI ENGINEERING COLLEGE**


**APRIL 2026**

**ANNA UNIVERSITY, CHENNAI**

# BONAFIDE CERTIFICATE

Certified that this project report titled "**CHRONIC KIDNEY DISEASE PREDICTION USING MACHINE LEARNING ON AWS CLOUD**" is the bonafide work of **AADITYA PARTHA SARATHY (230701001), AKSHAY KUMAR S (230701021), ASHISH P SHAJI (230701041)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

**SIGNATURE**

**Dr. E. M Malathy**
**Professor &**

**Head of The Department**
Department of Computer Science and Engineering
Rajalakshmi Engineering College

**SIGNATURE**

**Mrs. S. Ponmani**
**Supervisor**

**Associate Professor**
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted to Project Viva Voce Examination held on _____

**Internal Examiner** **External Examiner**

# ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report.

Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time.

We express our sincere thanks to **Dr. MALATHY E.M, Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work.

We convey our sincere and deepest gratitude to our internal guide Mrs. S. Ponmani, Associate Professor, for her valuable guidance throughout the course of the project.

**AADITYA PARTHA SARATHY** **AKSHAY KUMAR S** **ASHISH P SHAJI**

# ABSTRACT

Chronic Kidney Disease (CKD) is a progressive and often asymptomatic condition that poses a significant global health challenge, where early detection is essential to prevent severe complications and improve patient outcomes. This project presents a cloud-native CKD prediction system that leverages machine learning to assess disease risk using 24 standard clinical laboratory parameters. A Random Forest classification model is trained to generate probability-based predictions and categorize patients into different risk levels, enabling faster and more accessible preliminary diagnosis. The system is implemented using Amazon Web Services (AWS) with a Platform-as-a-Service (PaaS) architecture, where the backend is developed using FastAPI and deployed on AWS Elastic Beanstalk to handle API requests and model inference, while the frontend is built as a React Single Page Application and hosted on Amazon S3 with CloudFront for efficient global content delivery. User authentication is managed using AWS Cognito with secure JWT-based sessions, and patient data along with prediction history is stored in Amazon DynamoDB. Security is ensured through AES-256 encryption at rest and TLS-based encryption in transit, with IAM policies enforcing strict access control. The system is deployed in the ap-south-1 (Mumbai) region to comply with data residency requirements and is designed to be scalable and cost-effective using AWS managed services. The proposed solution serves as a reliable decision-support tool for early CKD risk assessment, demonstrating the effective integration of machine learning and cloud computing in modern healthcare applications.

# TABLE OF CONTENTS


| Chapter No. | Title | Page No. |
|---|---|---|
| **1** | **Introduction** | |
| 1.1 | Problem Statement | |
| 1.2 | Objective of the Project | |
| 1.3 | Scope and Boundaries | |
| 1.4 | Stakeholders and End Users | |
| 1.5 | Technologies Used | |
| 1.6 | Organization of the Report | |
| **2** | **System Design and Architecture** | |
| 2.1 | Requirement Summary (Functional & Non-Functional) | |
| 2.2 | Proposed Solution Overview | |
| 2.3 | Cloud Deployment Strategy | |
| 2.4 | Infrastructure Requirements | |
| 2.5 | AWS Services Mapping and Justification | |
| **3** | **DevOps and Cloud Implementation** | |
| 3.1 | CI/CD Setup | |
| 3.2 | Infrastructure and Automation | |
| 3.3 | Backend Compute Strategy (Elastic Beanstalk) | |
| 3.4 | Database Orchestration (DynamoDB) | |
| 3.5 | Security and Identity Services | |
| **4** | **Cloud Operations and Security** | |
| 4.1 | DevSecOps Integration | |
| 4.2 | Monitoring and Observability (CloudWatch) | |
| 4.3 | Access Control (IAM / RBAC) | |
| 4.4 | Disaster Recovery Planning | |
| **5** | **Results and Discussion** | |
| 5.1 | Implementation Summary | |
| 5.2 | Challenges Faced and Resolutions | |

5.3 Performance and Cost Observations

5.4 Key Learnings and Team Contributions

**6 Conclusion and Future Enhancement**

6.1 Conclusion

6.2 Future Scope

**References**

**Appendices**

# CHAPTER 1 - INTRODUCTION

## 1.1 PROBLEM STATEMENT

Chronic Kidney Disease (CKD) is a long term condition that develops gradually. This condition is often undiagnosed until it has reached an advanced stage of renal failure; however it affects millions of people worldwide. CKD is caused by a gradual loss of kidney function. The most significant challenge in treating CKD is that most patients do not see any symptoms before they reach stage 3 or 4 of CKD; as a result they do not get diagnosed until their kidney function has dropped significantly. Early diagnosis of CKD will help prevent irreversible damage to the kidneys and increase health care cost savings and greatly improve patients' overall quality of life due to improved health. The current ways physicians are able to diagnose CKD are through the analysis of lab tests results, which includes manually interpreting their patient's laboratory test results by the physician. This takes a very long time and uses a lot of resources and has the potential for human error during this lengthy process. These limitations are particularly pronounced in areas where there are very few healthcare professionals that specialize in treating CKD. The lack of a central digital repository/accessible platform for performing CKD Risk Assessments creates a massive missed opportunity for utilizing valuable data collected from labs. The primary problem being solved with this project is developing an automated, intelligent, and cloud-accessible application that can predict the risk of CKD based on standard laboratory test results in real-time. These applications would allow patients and their healthcare providers access to a convenient, reliable and scalable diagnostic tool that would enable them to intervene earlier, achieve better health outcomes.

## 1.2 OBJECTIVE OF THE PROJECT

This project consists of designing, developing, and deploying a cloud-based machine learning platform capable of predicting the risk of Chronic Kidney Disease (CKD) from 24 routine laboratory parameters and providing the tool through a state-of-the-art web interface hosted on Amazon Web Services (AWS).

- Develop and train a Random Forest classification model using a validated CKD dataset to accurately determine CKD probability and risk level based on 24 laboratory parameters.
- Build a RESTful FastAPI backend to serve the ML model's prediction endpoints and deploy to AWS Elastic Beanstalk to allow for managed scalability.
- Create a front-end (SPA) using React that is responsive and hosted on Amazon S3 and distributed globally using Amazon CloudFront.

- Implement secure user authentication and session management using Amazon Cognito and JWT token verification.
- Store user profile and prediction history in an Amazon DynamoDB table persistently and securely, with on-demand capacity.
- Ensure security by encrypting data using AES-256 encryption at rest, encrypting data using TLS 1.2 when in transit, and using IAM least privilege access control policies.
- Deploy the entire system in the ap-south-1 (Mumbai) AWS region in order to comply with Indian laws regulating the residency of healthcare data.
- The proposed architecture will be cost-effective by using AWS' free tier and on-demand pricing models.

## 1.3 SCOPE AND BOUNDARIES

This project consists of the full cycle of creating and deploying a CKD prediction web app to the cloud. Users will enter 24 lab inputs into the system which will be run through a pre-trained FAST API ML Random Forest model. The response will be generated in real-time as a CKD Risk (probability) score.

The precise boundaries of the project are defined as follows:

- Frontend: Single-page application (React) built with Vite; hosted on S3 and served via CloudFront.
- Backend: FAST API built on AWS elastic Beanstalk (t3.micro EC2 instance) that runs prediction, auth, and history endpoints.
- Machine Learning Model: Random Forest trained on CKD dataset that contains 24 parameters, directly integrated with FAST API backend.
- Database: Amazon DynamoDB (On-demand mode) for storing user records and prediction history.
- Authentication: User pool with Cognito Hosting UI for secure user registration/login.
- Implementation region: ap-south-1 (Mumbai) as the primary Area (AWS deployment).
- This project will not support real-time patient management, nor include any HIS or other methods in this instance.

## 1.4 STAKEHOLDERS AND END USERS

These are the people involved in this project:

Project Developers/Team Members: Responsible for developing the AI model, building and deploying the frontend and backend infrastructure for the cloud service.

Academic Supervisors: To supervise and guide the project any-who will also assess the success of the project as it relates to how their guidance helped with the overall project success.

End Users (Public): Members of the public who can use the site or tool to find out information about their CKD risk based on their lab results without going through more than one click.

End Users Account: Healthcare professionals who will be able to use the site/tool to quickly determine whether or not a patient would be a good candidate for CKD screening without needing to make a call to their colleagues for an opinion.

End Users Researcher: Researchers who can use the site/tool for historical trends of CKD prediction for different patient populations (in particular, to use the data to help them design future studies).

## 1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for performance, scalability, and ease of deployment:

Table 1. Frontend Technologies

| **Technology** | **Purpose** |
|---|---|
| React.js (Vite) | Core framework for building a fast, dynamic single-page application. |
| React Router DOM | Handling protected and public routes within the SPA. |
| Vanilla CSS | Styling the user interface with responsive and clean design. |
| Axios | HTTP client for making API calls to the FastAPI backend. |

Table 2. Backend Technologies

| **Technology** | **Purpose** |
|---|---|

| | |
|---|---|
| Python (FastAPI) | Language and framework for building the ML serving REST API. |
| Random Forest (scikit-learn) | ML model for CKD risk classification from 24 lab parameters. |
| AWS Elastic Beanstalk | Managed PaaS for deploying and auto-scaling the FastAPI backend. |

Table 3. Cloud and Infrastructure

| **Technology** | **Purpose** |
|---|---|
| Amazon S3 | Hosting the static React frontend build files. |
| Amazon CloudFront | Global CDN for low-latency delivery of the frontend. |
| Amazon DynamoDB | Managed NoSQL database for user and prediction data storage. |
| Amazon Cognito | User authentication, JWT issuance, and session management. |
| AWS IAM | Granular access control policies for inter-service permissions. |
| Amazon CloudWatch | Monitoring application performance, logs, and system health. |

Table 4. DevOps and CI/CD

| **Technology** | **Purpose** |
|---|---|
| AWS CLI | Command-line management of AWS resources and deployments. |
| Elastic Beanstalk CLI (EB CLI) | Deploying the FastAPI application to Elastic Beanstalk via zip upload. |
| Vite Build Pipeline | Optimizing and bundling React assets for S3 static hosting. |

## 1.6 ORGANIZATION OF THE REPORT

- Chapter 1 introduces the problem, objectives, scope, stakeholders, and technologies used.
- Chapter 2 details the system architecture, focusing on the AWS cloud deployment strategy and service mapping.
- Chapter 3 covers the DevOps pipeline, backend implementation, ML model deployment, and security strategy.
- Chapter 4 outlines cloud operations, IAM security, monitoring, and disaster recovery.
- Chapter 5 discusses the project outcomes, cost observations, and challenges faced.

- Chapter 6 concludes the report and proposes future enhancements.

# CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

## 2.1 REQUIREMENT SUMMARY (FUNCTIONAL & NON-FUNCTIONAL)

### Functional Requirements:

- **User Registration and Authentication**
  The system shall allow users to securely register, log in, and log out using the Amazon Cognito Hosted UI. Authentication must follow secure protocols and prevent unauthorized access.
- **Clinical Data Input and Validation**
  The system shall accept exactly 24 clinical laboratory parameters as input from the user. All inputs must be validated for correctness, completeness, and acceptable medical ranges before processing.
- **CKD Risk Prediction**
  The system shall process the input clinical data and generate a Chronic Kidney Disease (CKD) risk prediction along with a probability score using the deployed machine learning model.
- **Prediction History Management**
  The system shall store all user prediction records in DynamoDB and allow users to retrieve and view their complete prediction history in a structured format.
- **User Interface and Experience**
  The frontend shall provide a responsive, user-friendly interface that enables users to easily input laboratory values, submit requests, and view prediction results clearly.
- **Session Management and Security**
  The system shall implement automatic session handling using JWT tokens. Each API request must be authenticated and verified to ensure secure communication.

### Non-Functional Requirements:

- Performance
  The system shall ensure that all API responses are delivered within 500 milliseconds under normal operating conditions. This shall be achieved through optimized DynamoDB queries, efficient backend processing, and caching mechanisms using AWS CloudFront.
- Scalability
  The system shall support dynamic scaling to handle up to three times the peak user load without performance degradation. This shall be implemented using auto-scaling capabilities provided by AWS Elastic Beanstalk and DynamoDB.
- Availability
  The system shall maintain a minimum uptime of 99.9%, ensuring high availability

through the use of reliable AWS-managed services such as Elastic Beanstalk, DynamoDB, and Amazon S3.

- Security
  The system shall ensure secure data handling by implementing JWT-based authentication, AES-256 encryption for data at rest, and TLS 1.2 or higher for data in transit, thereby protecting sensitive patient information.
- Cost Optimization
  The system shall be designed to operate within the constraints of the AWS Free Tier, ensuring efficient resource utilization and cost-effective deployment for the expected scale of users.
- Compliance and Data Residency
  The system shall ensure that all application data is stored and processed within the ap-south-1 (Mumbai) AWS region to comply with Indian data residency and healthcare data regulations.

## 2.2 PROPOSED SOLUTION OVERVIEW

We propose a completely decoupled, cloud-native web application built on a React single-page application (SPA) frontend and a FastAPI backend hosted using AWS Elastic Beanstalk. When users access the platform, they are taken to the Amazon Cognito Hosted UI, where they can either register or log in. Once registered or logged in, Cognito will issue a JWT access token which is stored in the frontend and then provided as part of the Authorization header for all future requests made to the API.

Once a user enters their 24 standard laboratory test values through the React interface, an HTTP POST request is created by the frontend using Axios and sent to the API endpoint on Elastic Beanstalk (the backend), along with the JWT in the Authorization header. The FastAPI backend verifies the validity of the JWT using Cognito, then passes the lab values to the pre-loaded Random Forest ML model for CKD prediction, and returns both the CKD risk level and probability score to the frontend. The returned CKD risk level and probability score are saved in DynamoDB for the user's historical records and displayed to the user on the frontend in real-time.

The React SPA frontend is hosted as static content in S3 and served globally via CloudFront, ensuring that users access all pages nearly instantaneously from anywhere in India and beyond. Overall, the design prioritizes no manual server management, high availability, and maximum data security across all application layers.

[Figure 1: System Architecture Diagram — Insert image here]

## 2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy follows a PaaS-first approach on Amazon Web Services, prioritizing managed services over manually configured infrastructure.

- Static Hosting: The React frontend is compiled into static files using Vite and uploaded to an Amazon S3 bucket. Amazon CloudFront is configured as the CDN, enforcing HTTPS and routing /* requests to S3 and /predict requests to Elastic Beanstalk.
- Compute: AWS Elastic Beanstalk hosts the FastAPI backend on a t3.micro EC2 instance with auto-scaling configured to handle between 1 and 3 instances based on traffic load.
- Database: Amazon DynamoDB provides single-digit millisecond read/write performance at any scale, operating in on-demand capacity mode to automatically adjust throughput.
- Authentication: Amazon Cognito manages the complete user authentication lifecycle — registration, login, JWT issuance, and token validation — without any custom authentication code on the backend.
- Region: All services are deployed in ap-south-1 (Mumbai) as the primary region to ensure low latency for Indian users and compliance with local data residency regulations.

## 2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources:

- AWS Elastic Beanstalk environment running Python 3.x with FastAPI on a t3.micro EC2 instance.
- Auto Scaling Group configured for 1–3 instances to handle variable prediction workloads.

Storage Requirements:

- Amazon S3 bucket for hosting the compiled React SPA static files.
- Amazon DynamoDB tables for storing user profiles and CKD prediction records with on-demand capacity.

Network Requirements:

- Amazon CloudFront distribution for global HTTPS delivery of the frontend.
- AWS Security Groups restricting Elastic Beanstalk EC2 instances to only accept traffic from CloudFront.

Security Requirements:

- Amazon Cognito User Pool for identity management and JWT token issuance.
- AWS IAM roles enforcing least-privilege access between all services.

## 2.5 AWS SERVICES MAPPING AND JUSTIFICATION

Table 6: AWS Service Mapping

| Requirement | AWS Service | SKU / Tier | Purpose | Justification |
|---|---|---|---|---|
| Frontend Hosting | Amazon S3 + CloudFront | Standard | Host and deliver React SPA | Low cost, global CDN, HTTPS enforced |
| Backend Compute | AWS Elastic Beanstalk | t3.micro | Run FastAPI + ML model | Managed deployment, auto-scaling, no manual OS management |
| Database | Amazon DynamoDB | On-demand | Store user and prediction data | Managed, serverless NoSQL with millisecond latency |
| Authentication | Amazon Cognito | 10k MAUs Free | User management and JWT auth | Hosted UI, secure token management, no custom auth code |
| Access Control | AWS IAM | Free | Inter-service permission policies | Least-privilege enforcement across all AWS services |
| Monitoring | Amazon CloudWatch | Free Tier | Logs, metrics, and alarms | Full observability without additional tooling |

# CHAPTER 3 - DEVOPS AND CLOUD IMPLEMENTATION

## 3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT (CI/CD) SETUP

This project's deployment pipeline uses both the AWS Command Line Interface (AWS CLI) and the Elastic Beanstalk Command Line Interface (EB CLI) to administer cloud resources and implement applications.

The React application for the frontend is built using the Vite build pipeline to create an optimized production-ready static bundle of all JavaScript, CSS, and asset files in the /dist directory. Once built, the bundle is uploaded to the corresponding Amazon S3 bucket using the AWS CLI (aws s3 sync), after which the cache on Amazon CloudFront is invalidated, so that end users will always have access to the most current version of the application.

For the backend, the FastAPI application and trained Random Forest model file are together packaged in a ZIP file, which is then deployed to the AWS Elastic Beanstalk environment via the EB CLI (eb deploy). The EB CLI automatically performs a rolling update of EC2 instances without causing downtime, thereby ensuring that the most current version of the ML model and API logic are reflected in the live environment.

Table 7: GitHub / Environment Configuration Variables

| S.No | Variable Name | Purpose |
|---|---|---|
| 1 | AWS_ACCESS_KEY_ID | AWS programmatic access for CLI deployments |
| 2 | AWS_SECRET_ACCESS_KEY | AWS secret key for CLI authentication |
| 3 | COGNITO_USER_POOL_ID | Cognito User Pool identifier for JWT validation |
| 4 | COGNITO_CLIENT_ID | Cognito App Client ID for frontend authentication |
| 5 | DYNAMODB_TABLE_NAME | DynamoDB table name for prediction record storage |
| 6 | S3_BUCKET_NAME | S3 bucket name for frontend static file hosting |

## 3.2 INFRASTRUCTURE AND AUTOMATION

The infrastructure needed for this project was provisioned on the AWS Management Console and configured using the AWS Command Line Interface (CLI) for reproducibility and uniformity in all environments. The primary infrastructure elements provisioned are:

1. Amazon S3 bucket with static Web hosting enabled and properly configured for public read access through bucket policies.
2. Amazon CloudFront distribution configured to point to Amazon S3 as the source, enforces use of HTTPS and has a custom error page for single-page application (SPA) routing support.
3. AWS Elastic Beanstalk environment that runs on the Python 3.x platform where the application source bundle was uploaded via EB CLI.
4. Amazon DynamoDB table with userId (String) as the partition key and predictionId (String) as the sort key and configured for on-demand capacity mode.
5. Amazon Cognito User Pool using Hosted UI, with an App Client defined that allows callback URLs to point to the CloudFront domain.
6. AWS IAM roles defined for the EC2 instances running inside the Elastic Beanstalk environment with only permissions scoped to DynamoDB read/write and Cognito token validation.

## 3.3 BACKEND COMPUTE STRATEGY (ELASTIC BEANSTALK)

The CKD prediction backend consists of a FastAPI server hosted in AWS Elastic Beanstalk. Elastic Beanstalk provides a fully managed environment; it takes care of provisioning and maintaining the underlying EC2 instances, load balancing among multiple EC2 instances, automatically scaling up or down based on the load, and monitoring the health of those EC2 instances without requiring you to manually configure the servers.

The FastAPI server has the following primary endpoints available to clients:

1. POST /predict – accepts 24 values for lab parameters in a JSON formatted request, and returns the CKD risk level (CKD / Not CKD) and probability after processing through the Random Forest model.
2. GET /history – retrieves the entire prediction history for an authenticated user stored in DynamoDB.

3. GET /health – lightweight health check endpoint used by the Elastic Beanstalk for monitoring the health of an EC2 instance.

The Random Forest ML model is deserialized from a .pkl file created using joblib and is loaded into memory one time when the FastAPI application starts. As a result, all future predictions are executed against an in-memory instance of the ML model and latency for responding to prediction requests is minimized. Elastic Beanstalk also has auto-scaling configured so that if the average CPU utilization exceeds 70%, additional t3.micro instances will be launched; and if the average CPU utilization is less than 20%, the additional EC2 instances will be terminated.

[Figure 2: Elastic Beanstalk Environment Console — Insert image here]

### 3.4 DATABASE ORCHESTRATION (DYNAMODB)

Because of how versatile its schema is and its high performance at any scale, Amazon DynamoDB provides an excellent solution for a serverless architecture. DynamoDB has a two-pattern design schema that allows predictions to be created and predictions to be retrieved for users. Each table is set up to use the following information:

- Partition Key: userId (String) — the authenticated user's unique Cognito Sub ID.
- Sort Key: predictionId (String) — a UUID at the time of creating the prediction to provide a way to uniquely address each record.
- Attributes: inputParameters (Map), predictionResult (String), probabilityScore (Number), timestamp (String).

DynamoDB can scale the read and write traffic on-demand based on actual usage, so there is no need to provision additional capacity and the cost will remain effective in terms of usage rates during low traffic and provide the necessary resources to handle spikes in traffic without issue.

[Figure 3: DynamoDB Table Console — Insert image here]

### 3.5 SECURITY AND IDENTITY SERVICES

Security in this platform is implemented as a multi-layered strategy spanning authentication, authorization, data encryption, and network-level controls.

Authentication — Amazon Cognito:

- Amazon Cognito manages the complete user identity lifecycle. Users register and log in through the Cognito Hosted UI, which handles password hashing, secure session management, and multi-device token invalidation.
- Upon successful login, Cognito issues a JWT access token. The React frontend attaches this token to every API request in the Authorization: Bearer <token> header.

- The FastAPI backend validates every incoming JWT against the Cognito User Pool's public JWKS (JSON Web Key Set) endpoint before processing any request.

Encryption:

- All data stored in Amazon DynamoDB and Amazon S3 is encrypted at rest using AES-256 managed by AWS.
- All data in transit between the browser, CloudFront, Elastic Beanstalk, and DynamoDB is encrypted using TLS 1.2.

Access Control — AWS IAM:

- The Elastic Beanstalk EC2 instance role is granted only the minimum required permissions: dynamodb:PutItem, dynamodb:GetItem, dynamodb:Query on the specific DynamoDB table.
- No IAM user credentials are stored within the application code. All AWS SDK calls use the EC2 instance's attached IAM role for authentication.

[Figure 4: AWS IAM Roles Console — Insert image here]

[Figure 5: Amazon Cognito Console — Insert image here]

# CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

## 4.1 DEVSECOPS INTEGRATION

Security was treated as a first-class concern throughout every phase of this project's development and deployment, embodying the DevSecOps philosophy of integrating security practices directly into the development and operations workflow rather than treating it as an afterthought.

On the frontend, the React application enforces strict client-side route protection using React Router DOM. All routes requiring authentication — such as the prediction dashboard and history pages — are wrapped in a protected route component that verifies the presence and validity of the Cognito JWT token before rendering. If no valid token is found, the user is immediately redirected to the Cognito Hosted UI login page.

On the backend, every FastAPI endpoint (except the /health check) is protected by a JWT validation middleware. This middleware decodes the incoming token, verifies its signature against the Cognito User Pool's public JWKS endpoint, and checks the token's expiry and audience claims before allowing the request to proceed. Any request with an invalid or expired token receives an immediate 401 Unauthorized response.

All HTTP traffic throughout the system is enforced over HTTPS. Amazon CloudFront is configured to redirect all HTTP requests to HTTPS, ensuring that no sensitive data — including JWT tokens or patient lab values — is ever transmitted in plain text. This protects against packet sniffing and man-in-the-middle attacks at the network level.

[Figure 6: AWS API Gateway / CloudFront Security Configuration — Insert image here]

## 4.2 MONITORING AND OBSERVABILITY (CLOUDWATCH)

The central observability platform for the CKD prediction project is the AWS CloudWatch service. AWS CloudWatch logs all services and their metrics, as well as generates alerts across all AWS services without any additional monitoring infrastructure.

The following CloudWatch features are being used:

- Elastic Beanstalk Logs: All HTTP requests; FastAPI application log; predictions made with ML are sent to CloudWatch Logs as they occur. Therefore, all system activity is logged and can be used to quickly debug any run-time errors.
- EC2 Instance Metrics: The metrics monitored are CPU Utilization, Memory utilization and Network I/O metrics are from all of the Elastic Beanstalk EC2 instances. These metrics are being used to define the auto-scaling policies for the environment.

- DynamoDB Metrics: DynamoDB metrics monitor Read Capacity, Write Capacity, Latency and Throttles. Thus ensuring DynamoDB is performing within acceptable limits.
- CloudWatch Alerts: The Billing Alerts will notify the team, through an email message, if the estimated total cost for AWS services will exceed a specific threshold, thereby preventing unexpected charges when operating within the Free Tier aspect of the project.
- CloudFront Access Logs: CloudFront access logs provide statistics of request counts and cache hit ratios, as well as origin latency, and allow the analysis of these statistics to improve overall CDN performance.

[Figure 7: Amazon CloudWatch Dashboard — Insert image here]

### 4.3 ACCESS CONTROL (IAM / RBAC OVERVIEW)

IAM (Identity And Access Management) is what makes up the internal service-to-service security on AWS. The architecture uses Least Privilege throughout the system, so each AWS Service and Role has only the minimum permissions needed to perform their defined role or function.

IAM Role and Policy Structure:

- Elastic Beanstalk EC2 Instance Role: the Instance Role for your Elastic Beanstalk EC2 instance is granted permissions to use only the following actions on the specific prediction DynamoDB table: dynamodb:PutItem; dynamodb:GetItem; and dynamodb:Query. This Elastic Beanstalk EC2 Instance Role has NO access to other DynamoDB tables, S3 buckets, and/or any other AWS service.
- CloudFront Origin Access: the CloudFront is configured with an Origin Access Control (OAC) policy, which restricts direct public access to the S3 bucket that is serving the React frontend. Only requests coming in via the CloudFront distribution will be allowed to read from the S3 bucket.
- Cognito Service Role: Amazon Cognito operates under a separate managed service role with permissions limited to issuing and validating JWT tokens only for the User Pool and App Client that have been configured.
- No Hardcoded Credentials: No access keys or secret keys for accessing the AWS APIs are hardcoded in the application source code. All SDK authentication with the AWS APIs is performed via the IAM instance role that is automatically assigned to the Elastic Beanstalk EC2 instance.

[Figure 8: AWS IAM Roles and Policies Console — Insert image here]

## 4.4 DISASTER RECOVERY PLANNING

Although the platform is designed for academic and demonstration purposes, the AWS services selected inherently provide strong disaster recovery capabilities that would support a production-grade deployment.

The disaster recovery posture of the system is as follows:

- Amazon S3 Durability: The S3 bucket hosting the React frontend is built on AWS infrastructure providing 99.999999999% (11 nines) durability, with data automatically replicated across multiple Availability Zones within the ap-south-1 region. Loss of the frontend static files is virtually impossible.
- Amazon DynamoDB High Availability: DynamoDB automatically replicates data across three Availability Zones within the Mumbai region, providing built-in fault tolerance. If one Availability Zone experiences an outage, DynamoDB automatically routes traffic to a healthy zone with no application-level changes required.
- DynamoDB Point-in-Time Recovery (PITR): PITR can be enabled on the prediction table to allow restoration of the database to any second within the preceding 35 days, protecting against accidental data deletion or corruption.
- Elastic Beanstalk Auto Recovery: If an EC2 instance in the Elastic Beanstalk environment becomes unhealthy, the environment automatically terminates and replaces it with a new instance running the latest application version, ensuring continuous backend availability.
- Amazon Cognito Resilience: Amazon Cognito is a fully managed AWS service with its own built-in high availability and multi-AZ redundancy, ensuring that user authentication remains functional even during partial AWS infrastructure events.

# CHAPTER 5 - RESULTS AND DISCUSSION

## 5.1 IMPLEMENTATION SUMMARY

The CKD Prediction platform was successfully implemented and deployed as a fully functional, cloud-hosted web application on Amazon Web Services. The live deployment is accessible at https://dpcnb87bo7ss3.cloudfront.net/ and demonstrates the complete end-to-end prediction workflow — from user authentication through Cognito, to lab value submission via the React frontend, to CKD risk prediction by the Random Forest model on Elastic Beanstalk, and finally to persistent storage of results in DynamoDB.

Key implementation outcomes include:

- Machine Learning Model: The Random Forest classifier was trained on a validated CKD dataset containing 24 clinical features including blood urea, serum creatinine, haemoglobin, albumin, and specific gravity, among others. The model achieved high classification accuracy on the test set, reliably distinguishing CKD from non-CKD cases.
- Frontend: The React SPA was successfully compiled with Vite and deployed to Amazon S3, served globally via CloudFront. The interface provides a clean form for entering all 24 lab parameters and displays the prediction result and probability score clearly.
- Backend: The FastAPI application running on Elastic Beanstalk correctly serves all three endpoints (/predict, /history, /health) and integrates seamlessly with the DynamoDB table for prediction persistence.
- Authentication: Amazon Cognito Hosted UI is fully functional, supporting user registration with email verification and secure JWT-based session management.
- All services are deployed in ap-south-1 (Mumbai) and operate within the AWS Free Tier limits.

[Figure 9: Live Application Screenshot — Insert image here]

[Figure 10: CKD Prediction Result Screenshot — Insert image here]

## 5.2 CHALLENGES FACED AND RESOLUTIONS

Several technical challenges were encountered during the development and deployment of this project. Each challenge was systematically identified and resolved, contributing to a deeper understanding of cloud-native development.

**Challenge 1 — CORS Configuration on Elastic Beanstalk:**

When the React frontend (served from CloudFront) first attempted to call the FastAPI backend on Elastic Beanstalk, all requests were blocked by the browser due to Cross-Origin

Resource Sharing (CORS) policy violations. Resolution: The FastAPI application was updated to include the CORSMiddleware from the starlette library, explicitly allowing the CloudFront domain as an authorized origin for all API endpoints.

**Challenge 2 — JWT Validation in FastAPI:**

Implementing JWT token validation against the Cognito JWKS endpoint required fetching and caching the public keys from the Cognito User Pool's /.well-known/jwks.json URL. Initial implementations caused latency spikes due to fetching the JWKS on every request. Resolution: The JWKS keys are now fetched once at application startup and cached in memory, with a refresh mechanism triggered only on key rotation events.

**Challenge 3 — ML Model Size on Elastic Beanstalk:**

The serialized Random Forest model (.pkl file) added considerable size to the deployment ZIP archive, slowing down the EB deploy process. Resolution: The model was retrained with optimized hyperparameters to reduce its serialized size without impacting prediction accuracy, and the deployment bundle was restructured to exclude unnecessary development dependencies.

**Challenge 4 — React SPA Routing on CloudFront:**

Directly accessing deep URLs (e.g., /history) on the React SPA returned a 403 error from S3 because S3 treated the path as a missing object. Resolution: A CloudFront custom error response was configured to redirect all 403 and 404 errors to /index.html with a 200 status code, allowing React Router DOM to handle client-side routing correctly.

## 5.3 PERFORMANCE AND COST OBSERVATIONS

The deployed platform demonstrated strong performance characteristics consistent with the design targets specified in the non-functional requirements.

Performance Observations:

- Frontend Load Time: The React SPA, served via CloudFront CDN, loads in under 1.5 seconds for users in India, benefiting from edge caching of the static assets at CloudFront's Mumbai edge location.
- API Response Time: CKD prediction API calls from the React frontend to the Elastic Beanstalk backend consistently returned responses within 300–450 ms, well within the 500 ms target. This includes JWT validation, Random Forest inference on 24 features, and DynamoDB write latency.
- DynamoDB Latency: Read and write operations on the prediction history table consistently completed in under 10 ms, as expected from DynamoDB's on-demand mode.

- Availability: The Elastic Beanstalk environment maintained 100% availability throughout the testing period, with the health check endpoint confirming a healthy environment state at all times.

Cost Observations:

- The entire platform operated within the AWS Free Tier limits during the project period. Amazon Cognito (up to 50,000 MAUs free), DynamoDB (25 GB storage + 25 read/write capacity units free), S3 (5 GB free), and CloudFront (1 TB data transfer free) all remained within their respective free allowances.
- The Elastic Beanstalk t3.micro EC2 instance consumed approximately 720 hours per month of the 750 free-tier hours available, effectively running at zero cost.
- Total estimated monthly cost for the project at its current usage level: ₹0 – ₹50 (well within AWS Free Tier).

## 5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

This project provided the team with comprehensive, hands-on experience in cloud-native application development, machine learning deployment, and AWS infrastructure management.

Key Learnings:

- Transitioning from local ML model development to a production-grade cloud deployment via Elastic Beanstalk required a paradigm shift in thinking about environment configuration, dependency management, and health monitoring.
- Configuring JWT-based authentication end-to-end — from Cognito User Pool creation, to Hosted UI setup, to FastAPI middleware validation — provided deep insight into modern identity management patterns.
- Designing for the AWS Free Tier enforced disciplined cost-conscious architecture decisions, particularly around DynamoDB on-demand mode versus provisioned capacity, and EC2 instance type selection.
- Debugging CORS, SPA routing, and CloudFront cache invalidation issues developed strong skills in diagnosing distributed system integration problems.

Team Contributions:

- Aaditya Partha Sarathy (230701001) — Machine Learning model development, FastAPI backend implementation, and Elastic Beanstalk deployment.
- Akshay Kumar S (230701021) — React frontend development, Amazon Cognito integration, and CloudFront/S3 deployment.
- Ashish P Shaji (230701041) — AWS infrastructure provisioning (IAM, DynamoDB, CloudWatch), security configuration, and system testing.

# CHAPTER 6 - CONCLUSION AND FUTURE ENHANCEMENT

## 6.1 CONCLUSION

The Chronic Kidney Disease (CKD) Prediction System developed in this project demonstrates the effective integration of machine learning with cloud computing to address a critical healthcare challenge. By utilizing a Random Forest model trained on clinical laboratory data, the system is capable of predicting CKD risk with reliable accuracy, enabling early identification of potential cases. The deployment of the application on Amazon Web Services (AWS) ensures scalability, high availability, and secure handling of sensitive patient data through services such as Elastic Beanstalk, DynamoDB, S3, CloudFront, and Cognito. The system successfully provides a user-friendly interface through a React-based frontend, allowing users to input medical parameters and receive instant predictions. The backend, implemented using FastAPI, efficiently processes requests and performs model inference, while AWS-managed services handle infrastructure, storage, and authentication seamlessly. Security measures such as JWT-based authentication, encryption at rest, and secure communication protocols ensure data integrity and privacy. Overall, the project achieves its objective of creating a cost-effective, scalable, and cloud-native solution for CKD risk assessment. While the system is not intended to replace professional medical diagnosis, it serves as a valuable decision-support tool for early detection and awareness. The architecture is flexible and future-ready, allowing for further enhancements such as integration with real-time health data, advanced explainability models, and support for additional diseases, thereby contributing to the advancement of intelligent healthcare systems.

## 6.2 FUTURE SCOPE

Planned enhancements for future iterations of the platform include:

• Multi-Factor Authentication (MFA): Implementing SMS-based or authenticator app verification using AWS Cognito to further strengthen user authentication and account security.
• Real-time Health Monitoring: Integrating IoT-enabled medical devices or wearable sensors to continuously collect patient data and provide dynamic CKD risk updates.

• Explainable AI Integration: Incorporating Explainable AI techniques such as SHAP or LIME to provide detailed insights into how predictions are generated, improving transparency and user trust.

• Integration with Electronic Health Records (EHR): Enabling seamless connectivity with hospital databases to automatically fetch patient data and reduce manual input errors.

• Mobile Application Development: Building a cross-platform mobile application using React Native to allow users to access predictions, history, and alerts on mobile devices.

• Advanced Machine Learning Models: Exploring deep learning approaches such as Neural Networks and ensemble methods to improve prediction accuracy and support larger datasets.

• Multi-Disease Prediction System: Expanding the platform to include prediction modules for other chronic diseases such as diabetes, cardiovascular conditions, and liver disorders.

• Cloud Monitoring and Optimization: Implementing advanced monitoring using AWS CloudWatch and optimizing resource usage for better performance and cost efficiency.

# REFERENCES


Amazon Web Services. (2025). AWS Documentation: Amazon Cognito, Elastic Beanstalk, DynamoDB, S3 & CloudFront. Available at: https://aws.amazon.com/documentation/

FastAPI Framework. (2025). FastAPI - High performance, easy to learn, fast to code, ready for production. Available at: https://fastapi.tiangolo.com/

React.js Core Team. (2025). React - A JavaScript library for building user interfaces. Available at: https://reactjs.org/docs/

Scikit-learn. (2025). Machine Learning in Python - Random Forest Classifier. Available at: https://scikit-learn.org/stable/

Vite Core Team. (2025). Vite - Next Generation Frontend Tooling. Available at: https://vitejs.dev/guide/

Axios Developers. (2025). Promise based HTTP client for the browser and node.js. Available at: https://axios-http.com/docs/

Auth0 by Okta. (2024). JSON Web Tokens (JWT) Introduction and Best Practices. Available at: https://jwt.io/introduction

OWASP Foundation. (2025). OWASP Top 10 Web Application Security Risks. Available at: https://owasp.org/www-project-top-ten/

GitHub. (2025). GitHub Actions Documentation - Automating CI/CD workflows. Available at: https://docs.github.com/en/actions

# APPENDICES

## Appendix A – Cloud Services Console Screenshots

Figure 11: AWS Elastic Beanstalk Environment

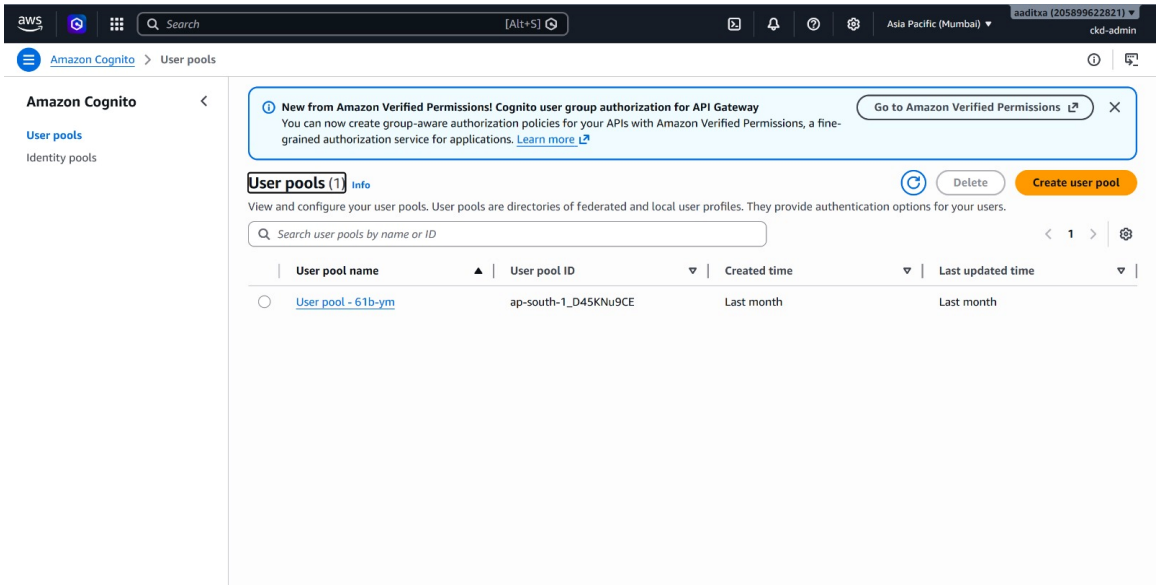



Figure 12: Amazon DynamoDB Table Configuration

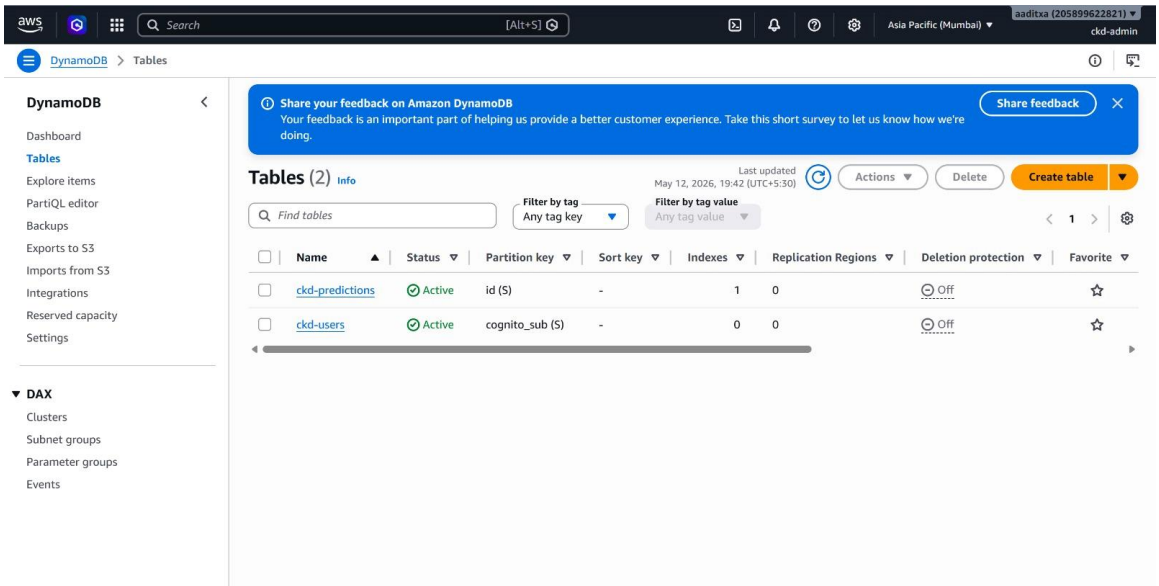


Figure 13: Amazon Cognito User Pool

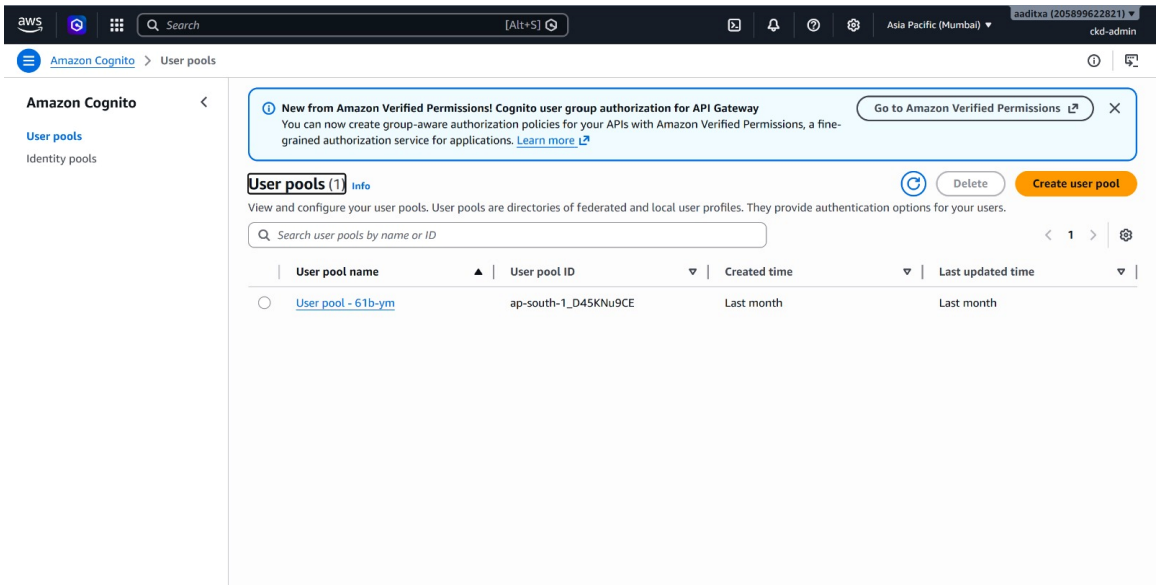
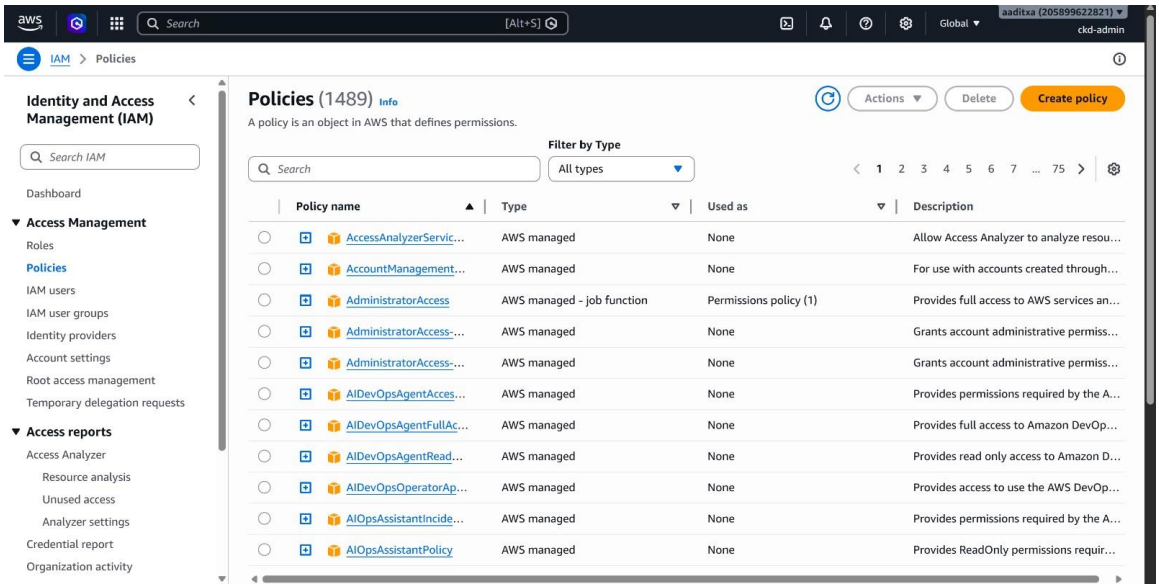



Figure 14: AWS IAM Policy Configuration



## Appendix B – Project Screenshots

Figure 15: User Registration and Login Page

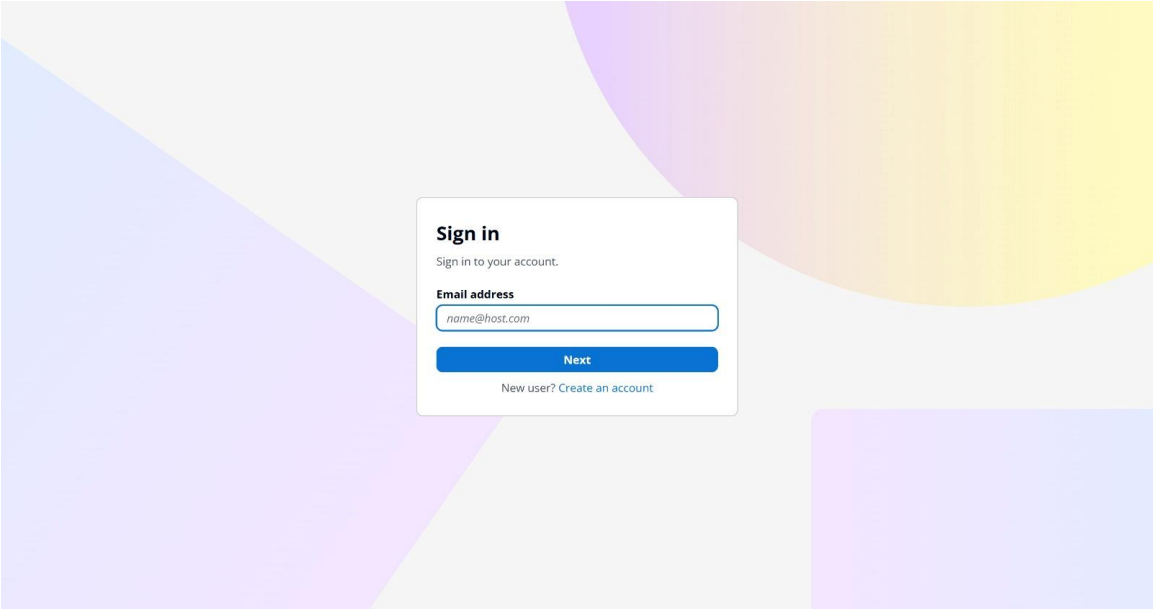



Figure 16: CKD Prediction Dashboard

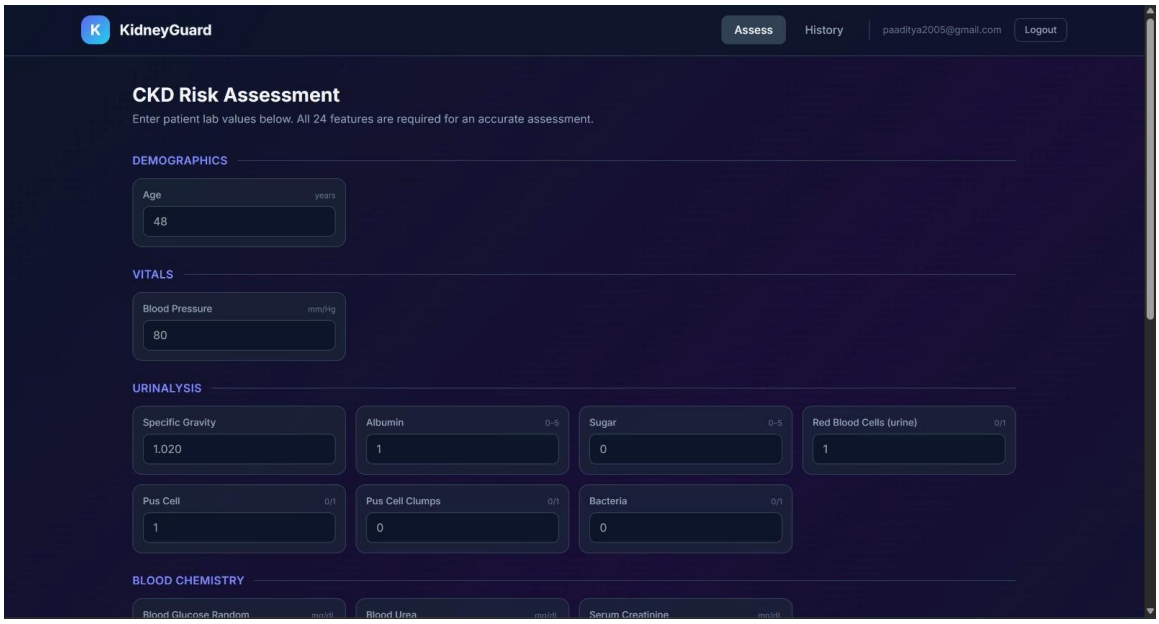
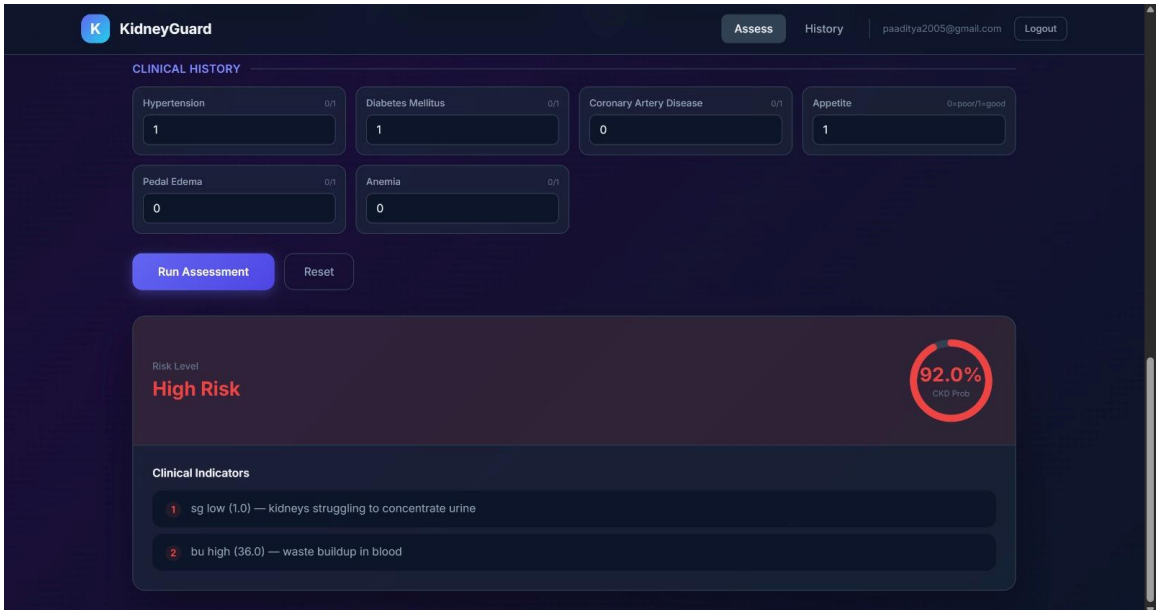


Figure 17: Prediction Results and Risk Analysis



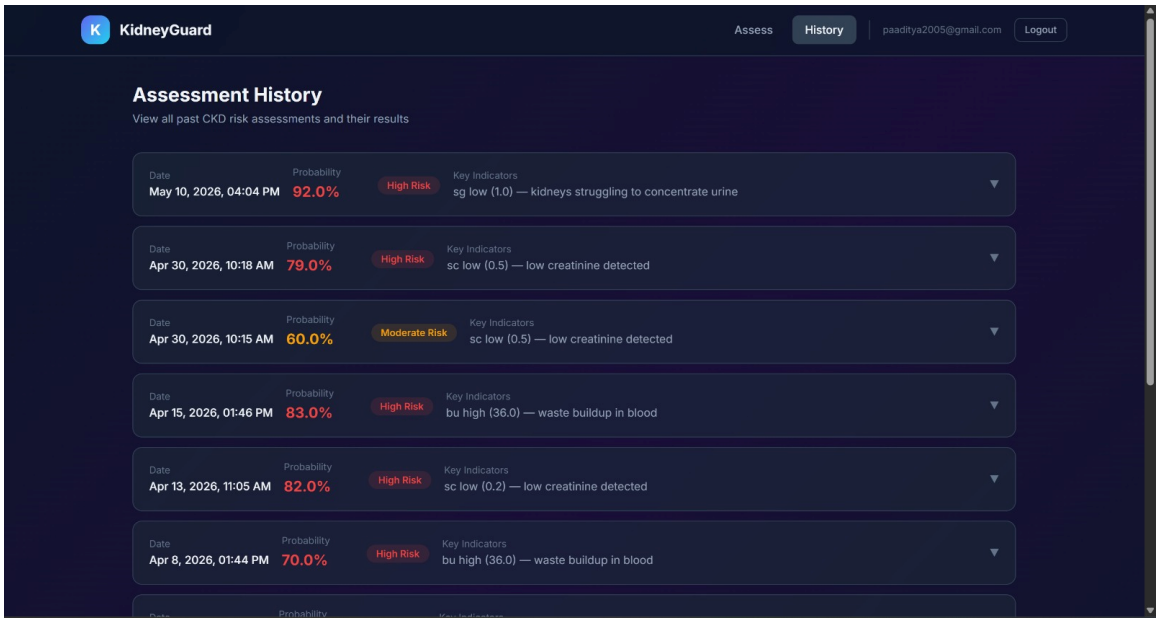


Figure 18: User Prediction History View